

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Технологии хранения данных»
Тема: RabbitMQ

Студент гр. 3342

Русанов А.И.

Преподаватель

Борисенко К.А.

Санкт-Петербург

2026

Цель работы

Создать docker-compose проект для демонстрации работы rabbitmq.

Описание решения

Приложение симулирует поток «гонок» и демонстрирует работу распределённой системы: продюсер генерирует события гонок и публикует их в RabbitMQ, два консьюмера читают и обрабатывают эти события в реальном времени.

Архитектура

- **Producer (FastAPI):** файл `router.py` — фоновый бесконечный цикл генерирует события для трёх гонок и публикует их в `direct exchange race_events` с `routing key`-ами `f1`, `nascar`, `lemans`.
- **RabbitMQ:** брокер с **Management UI** (контейнер `rabbitmq` в `docker-compose.yml`) — заменяет `Kafka + Zookeeper`, обеспечивает очереди, обменники и веб-интерфейс для мониторинга.
- **Consumers:**
 - `consumer1.py` — подписывается на очередь(и), привязанные к `exchange`, и выводит `live`-сообщения с метками/эмодзи.
 - `consumer2.py` — подписывается на те же очереди и отслеживает лидеров по каждой гонке, периодически выводя сводку.

Мониторинг

RabbitMQ Management UI (<http://localhost:15672>) используется для просмотра обменников, очередей и сообщений.

Поток данных

- **Producer** формирует JSON-сообщение:

```
{  
  "race": "F1",  
  "driver": "Hamilton",  
  "position": 1,  
  "lap": 50,  
  "timestamp": "..."  
}
```

- Producer публикует сообщение в exchange `race_events` с соответствующим `routing_key` (например, `f1`).
- Для обработки у каждого консьюмера существует `durable` очередь (например, `race_events_consumer1`, `race_events_consumer2`), привязанная к exchange по `routing keys` `f1`, `nascar`, `lemans`.
- При поступлении сообщения RabbitMQ доставляет его в привязанные очереди; консьюмеры получают сообщение, подтверждают его (ACK) и выполняют обработку (логирование, подсчёт лидеров и т.п.).

Скриншоты

1. Запуск и состояние контейнеров

```
aleksandrusanov@Aleksandrs-MacBook-Air rabbit-example % docker compose up --build
[+] Building 1.6s (27/27) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 1.58kB 0.0s
=> [consumer2 internal] load build definition from Dockerfile.consumer2 0.0s
=> => transferring dockerfile: 301B 0.0s
=> [consumer1 internal] load build definition from Dockerfile.consumer1 0.0s
=> => transferring dockerfile: 301B 0.0s
=> [app internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 304B 0.0s
=> [consumer1 internal] load metadata for mirror.gcr.io/library/python:3.11-slim 1.0s
=> [app internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [app 3/5] FROM mirror.gcr.io/library/python:3.11-slim@sha256:2c285c669cc837aa3bcf1ef23ea1932b7b6214f9c9d3aad22417446ad91cb4fb 0.0s
=> => resolve mirror.gcr.io/library/python:3.11-slim@sha256:2c285c669cc837aa3bcf1ef23ea1932b7b6214f9c9d3aad22417446ad91cb4fb 0.0s
=> [app internal] load build context 0.0s
=> => transferring context: 7.28kB 0.0s
=> [consumer2 internal] load build context 0.0s
=> => transferring context: 2.28kB 0.0s
=> [consumer1 internal] load build context 0.0s
=> => transferring context: 1.53kB 0.0s
=> [consumer2 2/5] WORKDIR /consumer 0.0s
=> [consumer1 3/5] COPY requirements.txt ./ 0.0s
=> [consumer1 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> [consumer1 5/5] COPY consumer/consumer1.py . 0.0s
=> [consumer2 3/5] COPY requirements.txt ./ 0.0s
=> [consumer2 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> [consumer2 5/5] COPY consumer/consumer2.py . 0.0s
=> [app 2/5] WORKDIR /app 0.0s
=> [app 3/5] COPY requirements.txt ./ 0.0s
=> [app 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> [app 5/5] COPY app . 0.0s
=> [app] exporting to image 0.1s
=> => exporting layers 0.0s
=> => exporting manifest sha256:1f6b408783da634aeaeaffc72cac5e25936573577582c1d2e33ba8a86a5ee575df 0.0s
=> => exporting config sha256:3aa085b32d6328312a19960639acb540d3b77da35d70285edf3fbb53b8e8e9a 0.0s
=> => exporting attestation manifest sha256:c919752cd9d72b81f899569fa566896a511b815261649a99303b571d8d52c1e4 0.0s
=> => exporting manifest list sha256:4e2160e8b06262705d7c74a27c71d6efb24a1dbddfbcca575ef980f972ebdd0 0.0s
=> => naming to docker.io/library/rabbit-example-app:latest 0.0s
=> => unpacking to docker.io/library/rabbit-example-app:latest 0.0s
=> [consumer2] exporting to image 0.1s
=> => exporting layers 0.0s
=> => exporting manifest sha256:6b5e7782331da4258a09ac9718a2d1b29baf99eb9fee855ba793481a4f9a24a3 0.0s
=> => exporting config sha256:3038a2f5d9f197f3de8e98031a27a33d8bc5c326a061855bb69a1ff6227f619f 0.0s
=> => exporting attestation manifest sha256:4e8d30739b22eda8d9da074e0990ee67f473d71358cc1728cab6b1dcd7307777 0.0s
=> => exporting manifest list sha256:79d558a434a47b4db1797ed6dfdb638e9d5aebaa2545260ff70ab3a2378c4b 0.0s
=> => naming to docker.io/library/rabbit-example-consumer2:latest 0.0s
=> => unpacking to docker.io/library/rabbit-example-consumer2:latest 0.0s
=> [consumer1] exporting to image 0.1s
=> => exporting layers 0.0s
=> => exporting manifest sha256:e319881d9e0216ef91c58c46439d58ec4e801b3f1f246e6698eeb186d1e02fd 0.0s
=> => exporting config sha256:b084094c854af5f6bb713c33f3e4f4e9e9a9a2d1b3d17b58847477320d07b653 0.0s
=> => exporting attestation manifest sha256:22f5eb867791cad27151a5695d87874e744f4b3b4fb57e8d81723924a1c511 0.0s
=> => exporting manifest list sha256:79825094e99a82a9fb720f348168a56cc496ed2dc6847ee0103ff539ec0c9a3e 0.0s
=> => naming to docker.io/library/rabbit-example-consumer1:latest 0.0s
=> => unpacking to docker.io/library/rabbit-example-consumer1:latest 0.0s
=> [app] resolving provenance for metadata file 0.0s
=> [consumer1] resolving provenance for metadata file 0.0s
=> [consumer2] resolving provenance for metadata file 0.0s
[+] up 7/7
✔ Image rabbit-example-app Built 1.7s
✔ Image rabbit-example-consumer1 Built 1.7s
✔ Image rabbit-example-consumer2 Built 1.7s
✔ Container rabbitmq Healthy 5.0s
✔ Container rabbit-consumer2 Started 5.9s
✔ Container rabbit-consumer1 Started 5.9s
✔ Container rabbit-app Started 5.9s
aleksandrusanov@Aleksandrs-MacBook-Air rabbit-example % docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
rabbit-app rabbit-example-app "/bin/sh -c 'uvicorn..." app 17 seconds ago Up 11 seconds 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
rabbit-consumer1 rabbit-example-consumer1 "/bin/sh -c 'python3..." consumer1 17 seconds ago Up 11 seconds
rabbit-consumer2 rabbit-example-consumer2 "/bin/sh -c 'python3..." consumer2 17 seconds ago Up 11 seconds
rabbitmq rabbitmq:3.13-management "docker-entrypoint.s..." rabbitmq 17 seconds ago Up 16 seconds (healthy) 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp
```

Результат docker compose up -d --build и вывод
docker compose ps

2. Просмотр логов consumer1

```
aleksandrusanov@Aleksandrs-MacBook-Air rabbit-example % docker compose logs -f consumer1
rabbit-consumer1 | Starting Consumer 1...
rabbit-consumer1 | [CONSUMER1] RACE F1 UPDATE: Hamilton at P1 on lap 1 @ 2026-05-20T11:56:24.544432
rabbit-consumer1 | [CONSUMER1] RACE F1 UPDATE: Verstappen at P2 on lap 1 @ 2026-05-20T11:56:24.548052
rabbit-consumer1 | [CONSUMER1] RACE F1 UPDATE: Sainz at P3 on lap 1 @ 2026-05-20T11:56:24.549193
rabbit-consumer1 | [CONSUMER1] RACE F1 UPDATE: Leclerc at P4 on lap 1 @ 2026-05-20T11:56:24.550523
rabbit-consumer1 | [CONSUMER1] USA NASCAR UPDATE: Johnson at P1 on lap 1 @ 2026-05-20T11:56:24.551423
rabbit-consumer1 | [CONSUMER1] USA NASCAR UPDATE: Elliott at P2 on lap 1 @ 2026-05-20T11:56:24.552254
rabbit-consumer1 | [CONSUMER1] USA NASCAR UPDATE: Logano at P3 on lap 1 @ 2026-05-20T11:56:24.553099
rabbit-consumer1 | [CONSUMER1] USA NASCAR UPDATE: Larson at P4 on lap 1 @ 2026-05-20T11:56:24.553936
rabbit-consumer1 | [CONSUMER1] FR LeMans UPDATE: Porsche Team at P1 on lap 1 @ 2026-05-20T11:56:24.554998
rabbit-consumer1 | [CONSUMER1] FR LeMans UPDATE: Ferrari Team at P2 on lap 1 @ 2026-05-20T11:56:24.555826
rabbit-consumer1 | [CONSUMER1] FR LeMans UPDATE: Toyota Team at P3 on lap 1 @ 2026-05-20T11:56:24.556656
rabbit-consumer1 | [CONSUMER1] FR LeMans UPDATE: BMW Team at P4 on lap 1 @ 2026-05-20T11:56:24.558057
rabbit-consumer1 | [CONSUMER1] RACE F1 UPDATE: Hamilton at P1 on lap 2 @ 2026-05-20T11:56:26.562324
rabbit-consumer1 | [CONSUMER1] RACE F1 UPDATE: Verstappen at P2 on lap 2 @ 2026-05-20T11:56:26.570525
rabbit-consumer1 | [CONSUMER1] RACE F1 UPDATE: Sainz at P3 on lap 2 @ 2026-05-20T11:56:26.573387
rabbit-consumer1 | [CONSUMER1] RACE F1 UPDATE: Leclerc at P4 on lap 2 @ 2026-05-20T11:56:26.575733
rabbit-consumer1 | [CONSUMER1] USA NASCAR UPDATE: Johnson at P1 on lap 2 @ 2026-05-20T11:56:26.577908
rabbit-consumer1 | [CONSUMER1] USA NASCAR UPDATE: Elliott at P2 on lap 2 @ 2026-05-20T11:56:26.579863
rabbit-consumer1 | [CONSUMER1] USA NASCAR UPDATE: Logano at P3 on lap 2 @ 2026-05-20T11:56:26.581782
rabbit-consumer1 | [CONSUMER1] USA NASCAR UPDATE: Larson at P4 on lap 2 @ 2026-05-20T11:56:26.584024
```

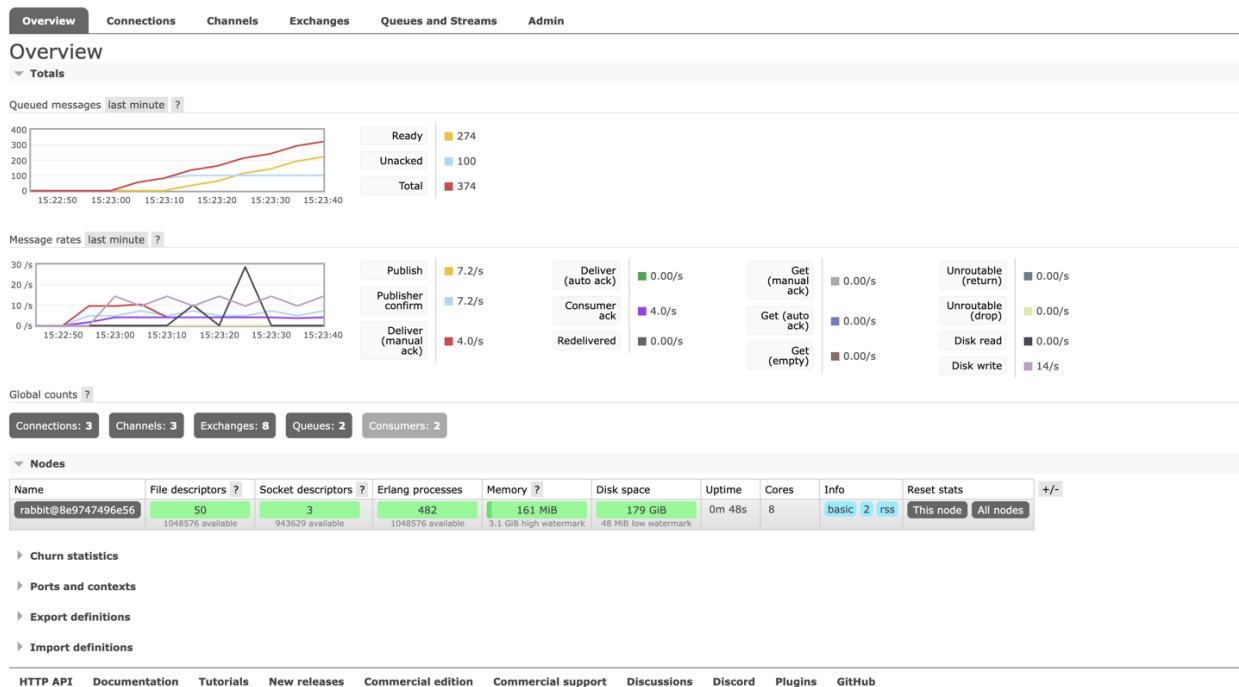
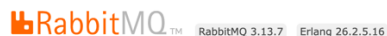
Поток логов consumer1 (docker compose logs -f consumer1)

3. Просмотр логов consumer2

```
aleksandrivanov@Aleksandr-MacBook-Air rabbit-example % docker compose logs -f consumer2
rabbit-consumer2 Starting Consumer 2 (Leaders Tracker)...
rabbit-consumer2
rabbit-consumer2 =====
rabbit-consumer2 [CONSUMER2] CURRENT LEADERS:
rabbit-consumer2 =====
rabbit-consumer2 F1      -> Hamilton      | Lap 1 | Messages: 4
rabbit-consumer2 NASCAR   -> Johnson     | Lap 1 | Messages: 4
rabbit-consumer2 LeMans   -> Porsche Team | Lap 1 | Messages: 4
rabbit-consumer2 =====
rabbit-consumer2
rabbit-consumer2 [CONSUMER2] CURRENT LEADERS:
rabbit-consumer2 =====
rabbit-consumer2 F1      -> Hamilton      | Lap 2 | Messages: 8
rabbit-consumer2 NASCAR   -> Johnson     | Lap 2 | Messages: 8
rabbit-consumer2 LeMans   -> Porsche Team | Lap 2 | Messages: 8
rabbit-consumer2 =====
rabbit-consumer2
rabbit-consumer2 [CONSUMER2] CURRENT LEADERS:
rabbit-consumer2 =====
rabbit-consumer2 F1      -> Hamilton      | Lap 3 | Messages: 12
rabbit-consumer2 NASCAR   -> Johnson     | Lap 3 | Messages: 12
rabbit-consumer2 LeMans   -> Porsche Team | Lap 3 | Messages: 12
rabbit-consumer2 =====
rabbit-consumer2
rabbit-consumer2 [CONSUMER2] CURRENT LEADERS:
rabbit-consumer2 =====
rabbit-consumer2 F1      -> Hamilton      | Lap 4 | Messages: 16
rabbit-consumer2 NASCAR   -> Johnson     | Lap 4 | Messages: 16
rabbit-consumer2 LeMans   -> Porsche Team | Lap 4 | Messages: 16
rabbit-consumer2 =====
rabbit-consumer2
rabbit-consumer2 [CONSUMER2] CURRENT LEADERS:
rabbit-consumer2 =====
rabbit-consumer2 F1      -> Hamilton      | Lap 5 | Messages: 20
rabbit-consumer2 NASCAR   -> Johnson     | Lap 5 | Messages: 20
rabbit-consumer2 LeMans   -> Porsche Team | Lap 5 | Messages: 20
rabbit-consumer2 =====
rabbit-consumer2
rabbit-consumer2 [CONSUMER2] CURRENT LEADERS:
rabbit-consumer2 =====
rabbit-consumer2 F1      -> Verstappen   | Lap 6 | Messages: 24
rabbit-consumer2 NASCAR   -> Johnson     | Lap 6 | Messages: 24
rabbit-consumer2 LeMans   -> BMW Team     | Lap 6 | Messages: 24
rabbit-consumer2 =====
```

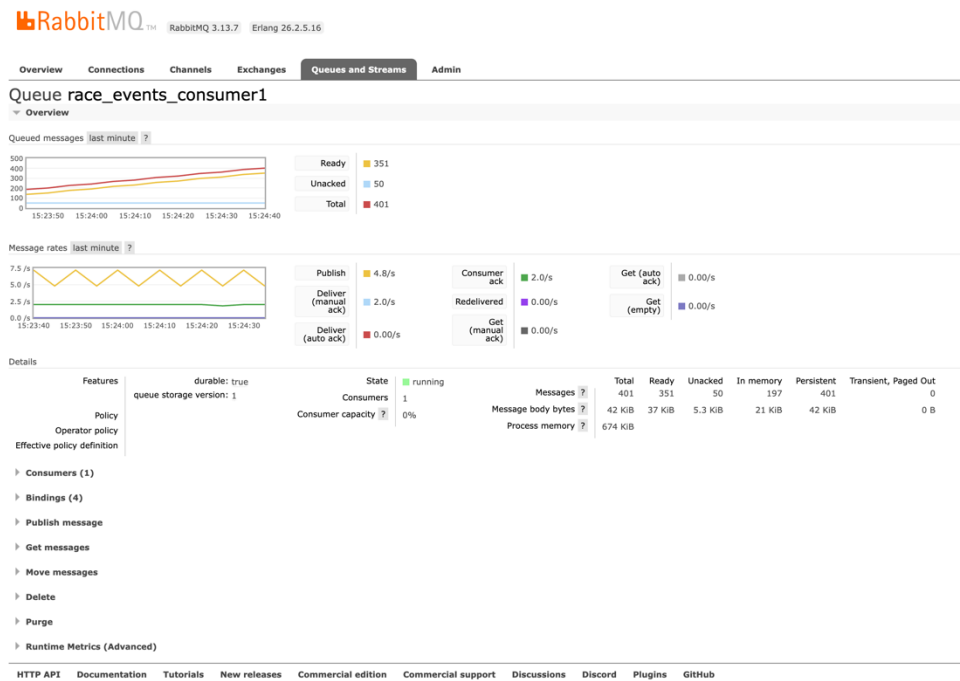
Поток логов consumer2 (docker compose logs -f consumer2)

4. RabbitMQ Management UI

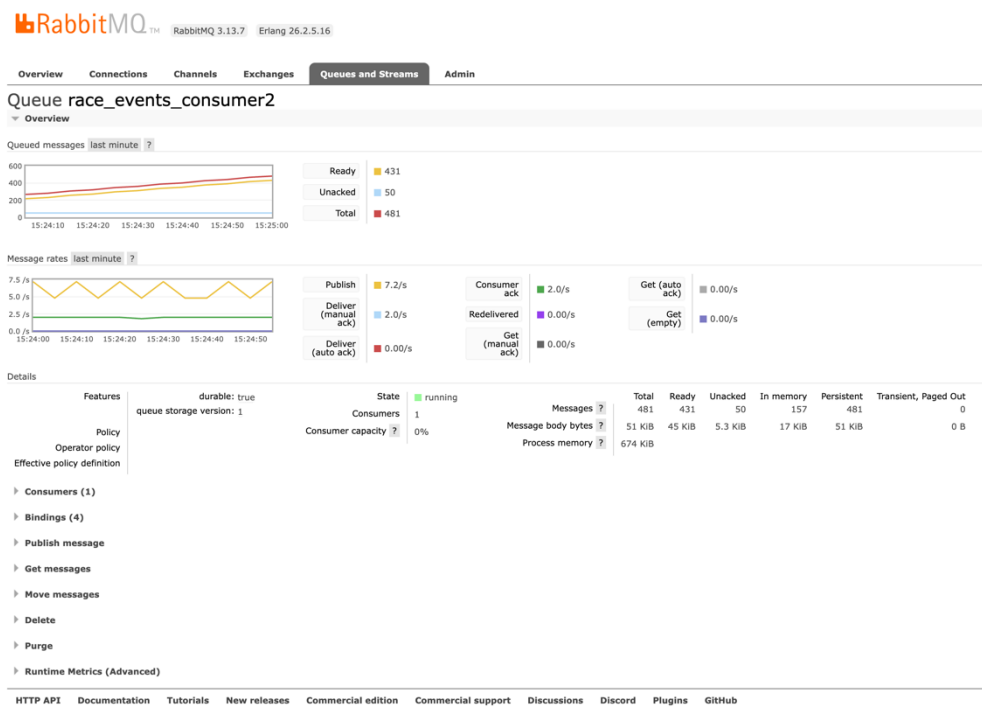


RabbitMQ Management UI — overview

5. RabbitMQ: очередь race_events_consumer1



6. RabbitMQ: очередь race_events_consumer2



Выводы

В ходе работы был создан и успешно запущен docker-compose проект для демонстрации работы RabbitMQ. Реализован producer на FastAPI, который генерирует и отправляет события о гоночных заездах в direct exchange race_events с routing key-ами fl, nascar, lemans, а также два consumer-а, обрабатывающих эти сообщения в реальном времени. Корректность работы системы подтверждается тем, что сообщения доставляются в RabbitMQ, читаются consumer-ами и отображаются в интерфейсе RabbitMQ Management (<http://localhost:15672>), что позволяет визуально контролировать состояние exchange'ов, очередей и поток данных.